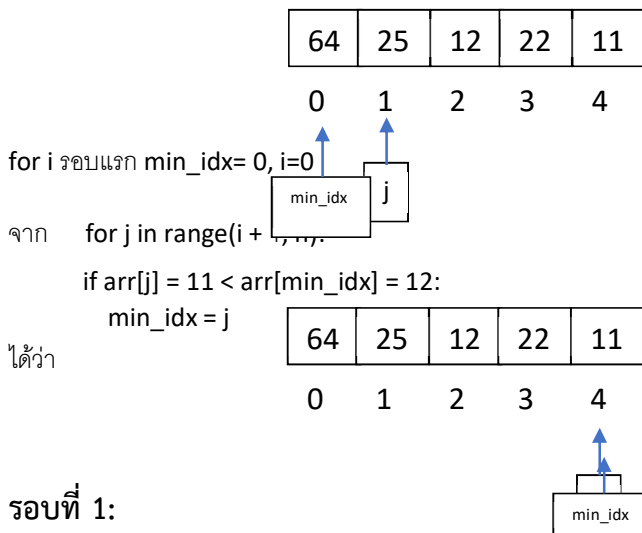


Example: Selection Sort

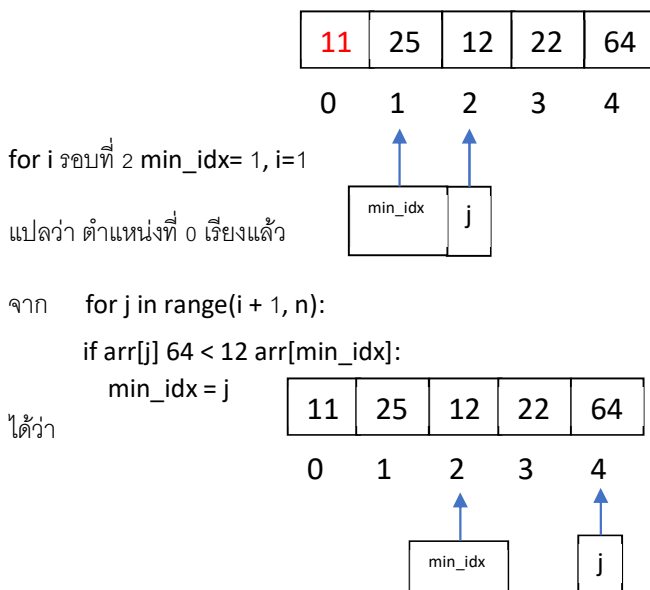
If there is an existing array (list in Python) that contains the numbers [64, 25, 12, 22, 11], write the steps for the selection_sort() function.

Each round consists of:

1. What the minimum value (and its index) is.
2. What the remaining unsorted elements are.

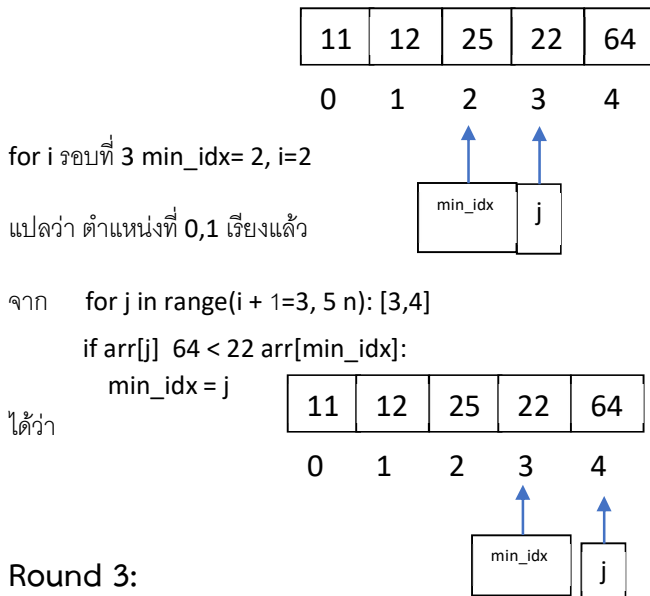


1. **หาค่าน้อยที่สุด:** ในอาร์เรย์ [64, 25, 12, 22, 11] ค่าน้อยที่สุดคือ **11** ซึ่งอยู่ที่ตำแหน่งดัชนี 4
2. **สลับ:** สลับค่าน้อยที่สุด (11) กับสมาชิกตัวแรกของส่วนที่ยังไม่ได้จัดเรียง (64)
 - ก่อนสลับ: [64, 25, 12, 22, 11]
 - หลังสลับ: [**11**, 25, 12, 22, 64]
3. **ผลลัพธ์:** สมาชิกตัวแรก (11) ถูกจัดเรียงแล้ว ตอนนี้ส่วนที่ยังไม่ได้จัดเรียงคือ [25, 12, 22, 64]



Round 2:

1. **Find the minimum** : From the unsorted array [25, 12, 22, 64] the minimum is **12** at index 2
2. **Swap**: swap the minimum (12) with the first of the unsorted array (25)
 - Before swap: [11, **25**, 12, 22, 64]
 - After swap : [11, **12**, **25**, 22, 64]
3. **Result**: Now, 2 elements (11, 12) was sorted. The remaining unsorted array is [25, 22, 64]



Round 3:

1. **Find the minimum** : From the unsorted array [25, 22 ,64] the minimum is **22** at index 3
2. **Swap**: swap the minimum **22** with the first of the unsorted array **25**
 - Before swap: [11, 12, **25**, 22, 64] เขียนสมาชิกใน array ให้ครบทั้ง 5 ตัว
 - After swap : [11, 12, **22**, **25**, 64] เขียนสมาชิกใน array ให้ครบทั้ง 5 ตัว
3. **Result**: Now, **3** elements (**11, 12, 22**) was sorted. The remaining unsorted array is [**25, 64**]

Round 4:

1. **Find the minimum** : From the unsorted array [**25, 64**] the minimum is **25** at index 3
2. **Swap**: swap the minimum **25** with the first of the unsorted array **25**
 - Before swap: [**11, 12, 22, 25, 64**] เขียนสมาชิกใน array ให้ครบทั้ง 5 ตัว

- After swap : [11, 12, 22, 25, 64] เขียนสมาชิกใน array ให้ครบทั้ง 5 ตัว

3. Result: Now, 4 elements (11, 12, 22, 25) was sorted. The remaining unsorted array is [64]

Round 5:

1. There is only one element [64] mean sorted already.

2. Result: Now, all elements were sorted [11, 12, 22, 25, 64].

[สรุปสาระสำคัญของ Selection Sort และเกร็ดความรู้ Python Internals]:

* สาระสำคัญและข้อควรระวังใน Selection Sort (Exam Cautions):

1. กฎเหล็ก: ห้ามข้ามขั้นตอนการเช็คของ Round 4 และ Round 5 แม้มันจะเห็นว่าเรียงแล้วก็ตาม!

- Round 4: min_idx = 3 (ค่า 25), min_idx == i (สลับกับตัวเอง ไม่มีการเปลี่ยนตำแหน่ง)

- Round 5: สมาชิกเหลือตัวเดียว [64] ถือว่าจัดเรียงเสร็จสมบูรณ์โดยอัตโนมัติ

2. ความหมายของคำสั่ง for j in range(i + 1, n): ตรวจสอบเฉพาะสมาชิกใน 'ส่วนที่ยังไม่ได้จัดเรียง'

3. range(0) ไม่ใช่ None และไม่ใช่ [0]! แต่เป็น Empty Range Object ที่ไม่มีการจองหน่วยความจำ

2

3

4

ไม่ใช่

1

2

3

4

5

ฟังก์ชัน range(n) เป็น ฟังก์ชันสร้างลำดับตัวเลข เช่น range(5) จะได้ผลลัพธ์เหมือนกับ [0,1,2,3,4]

range(5) ไม่ได้สร้างลิสต์หรืออาร์เรย์ของตัวเลขเตรียมไว้ล่วงหน้า แต่เก็บแค่ข้อมูลการสร้าง Metadata เพียง 3 ค่าสั้นๆ คือ:

- start = 0
- stop = 5
- step = 1

ดังนั้น for j in range(n) เป็นการเหมือนกับนำค่าใน [0,1,2,3,4] มาใส่ในตัวแปร j ในแต่ละรอบของ for loop

ทำความเข้าใจกับ `for j in range(n)`

ฟังก์ชัน `range(n)` เป็น ฟังก์ชันสร้างลำดับตัวเลขที่กินหน่วยความจำน้อยกว่า การเขียนเงื่อนไข

ฟังก์ชัน `range(start, stop)` ใน Python จะเริ่มนับตั้งแต่ค่า `start` ไปจนถึงก่อนถึงค่า `stop` (คือจบที่ `stop - 1`)

หลักการทำงานของ `range(5)`

`range(5)` ไม่ได้สร้างลิสต์หรืออาร์เรย์ของตัวเลขเตรียมไว้ล่วงหน้า แต่เก็บแค่ข้อมูลการสร้าง Metadata เพียง 3 ค่า
สั้นๆ คือ:

- `start = 0`
- `stop = 5`
- `step = 1`

ถ้าไม่ใช่ ฟังก์ชัน `range(n)` เราจะเขียนแทนด้วย `while loop` ดังนี้

ภาษา Python (ถ้าอยากให้ทำงานแบบเดิมแบบไม่มี `range` ต้องใช้ `while`)

```
j = 0
while j < n:
    # โค้ดที่ต้องการรัน
    j += 1
```

ฟังก์ชัน `range(n)` เป็น ฟังก์ชันสร้างลำดับตัวเลข เช่น `range(5)` จะได้ผลลัพธ์เหมือนกับ `[0,1,2,3,4]`

`range(5)` ไม่ได้สร้างลิสต์หรืออาร์เรย์ของตัวเลขเตรียมไว้ล่วงหน้า แต่เก็บแค่ข้อมูลการสร้าง Metadata เพียง 3 ค่า
สั้นๆ คือ:

- `start = 0`
- `stop = 5`
- `step = 1`

ดังนั้น `for j in range(n)` เป็นการเหมือนกับนำค่าใน `[0,1,2,3,4]` มาใส่ในตัวแปร `j` ในแต่ละรอบของ `for loop`

ถามว่า `range(0)` มีค่าเป็น `None` หรือ `[0]`

คำตอบคือ **ไม่มีทั้งคู่** ครับ

`range(0)` ไม่ใช่ `None` และไม่ใช่ `[0]` แต่เป็น ลำดับว่างเปล่า (Empty Range/Sequence) ที่มีจำนวนสมาชิก
เป็น 0 ตัวครับ

range(0) มีความหมายเดียวกับ [] ใช่หรือไม่

ทางผลลัพธ์การใช้งาน ถือว่าเหมือนกันครับ แต่ ในเชิงโครงสร้างข้อมูล ไม่เหมือนกัน

1. ส่วนที่เหมือนกัน (เมื่อนำไปใช้งาน)

- ผลลัพธ์ในลูป for: ทั้งคู่จะ ไม่ทำงานเลยแม้แต่รอบเดียว (ข้ามลูปทันที)

```
for j in range(0):
```

```
    print("A") # ไม่พิมพ์
```

```
for j in []:
```

```
    print("A") # ไม่พิมพ์
```

2. ส่วนที่ต่างกัน (ชนิดของข้อมูล)

- [] คือ List (จองพื้นที่ในหน่วยความจำเพื่อเก็บข้อมูล)
- range(0) คือ Range Object (ตัวสร้างลำดับตัวเลขที่กินหน่วยความจำน้อยกว่า ไม่เกิดการจองพื้นที่ในหน่วยความจำ ไม่เกิดลิสต์)

แล้ว for j in range(i + 1, n): หมายถึงอย่างไร ใน Selection Sort

คำสั่ง for j in range(i + 1, n): หมายถึง "การให้ j วนลูปดูทีละตัว ตั้งแต่ตำแหน่งถัดจาก i ไปจนถึงตัวสุดท้ายของอาร์เรย์"

ฟังก์ชัน range(start, stop) ใน Python จะเริ่มนับตั้งแต่ค่า start ไปจนถึงก่อนถึงค่า stop (คือจบที่ stop - 1)

มองภาพให้เห็นง่ายขึ้นด้วย Selection Sort

สมมติมีข้อมูล 5 ตัว (n = 5 ดัชนีคือ 0, 1, 2, 3, 4)

- เมื่อ i = 0 (รอบแรก):
 - for j in range(i + 1, n) จะกลายเป็น range(1, 5)
 - j จะวิ่งไปที่ตำแหน่ง 1, 2, 3, 4
 - ความหมาย: เราสมมติให้ตัวที่ 0 น้อยที่สุดก่อน (min_idx = 0) แล้วใช้ j เดินสำรวจตัวที่เหลือทั้งหมดเพื่อหาว่ามีตัวไหนน้อยกว่าตัวที่ 0 ไหม

- เมื่อ $i = 1$ (รอบที่สอง):
 - ตำแหน่งที่ 0 เรียงเสร็จแล้ว ไม่ต้องยุ่งกับมันอีก
 - `for j in range(i + 1, n)` จะกลายเป็น `range(2, 5)`
 - `j` จะวิ่งไปที่ตำแหน่ง 2, 3, 4
 - **ความหมาย:** เราเปลี่ยนไปสำรวจเฉพาะกลุ่มที่ยังไม่ได้เรียง (ตั้งแต่นำหน้า i ไปจนจบ)

หน้าต่างประโยคคำสั่งที่ main

```
my_list = [64, 25, 12, 22, 11]
selection_sort(my_list)
print("Result : ", my_list)
```